

Higher Nationals - Summative Assignment Feedback Form

Student Name/ID			
Unit Title	Software Development Lifecycles		
Assignment	Assignment 01	Assessor	Ms. ALF. Sajeetha
Submission Date	08.12.2024	Date Received 1st	
Re-submission Date		Date Received 2nd submission	
Assessor Feedback: *Please note that constructive and useful feedback should allow students to understand: a) Strengths of performance b) Limitations of performance c) Any improvements needed in future assessments Feedback should be against the learning outcomes and assessment criteria to help students understand how these inform the process of judging the overall grade. Feedback should give full guidance to the students on how they have met the learning outcomes and assessment criteria.			
<i>I certify that to the best of my knowledge the evidence submitted for this assignment is the student's own. The student has clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I have not solely used AI to grade the student's work.</i>			
Grade:	Assessor Signature:		Date:
Resubmission Feedback: *Please note resubmission feedback is focussed only on the resubmitted work			
Grade:	Assessor Signature:		Date:
Internal Verifier's Comments:			
Signature & Date:			

* Please note that grade decisions are provisional. They are only confirmed once internal and external moderation has taken place and grades decisions have been agreed at the assessment board.

STUDENT ASSESSMENT SUBMISSION AND DECLARATION

When submitting evidence for assessment, each student must sign a declaration confirming that the work is their own.

Student name: MN. Sihanass / 1030371		Assessor name: Ms. ALF. Sajeetha	
Issue date: 24.09.2024	Submission date: 08.12.2024	Submitted on: 08.12.2024	
Programme: HND in Computing			
Unit: Software Development Lifecycle			
Assignment number and title: System for EaseShop Central (PVT) Ltd			

Plagiarism

Plagiarism is a particular form of cheating. Plagiarism must be avoided at all costs and students who break the rules, however innocently, may be penalised. It is your responsibility to ensure that you understand correct referencing practices. As a university level student, you are expected to use appropriate references throughout and keep carefully detailed notes of all your sources of materials for material you have used in your work, including any material downloaded from the Internet. Please consult the relevant unit lecturer or your course tutor if you need any further advice.

Student Declaration

Student declaration

I certify that the assignment submission is entirely my own work and I fully understand the consequences of plagiarism. I declare that the work submitted for assessment has been carried out without assistance other than that which is acceptable according to the rules of the specification. I certify I have clearly referenced any sources and any artificial intelligence (AI) tools used in the work. I understand that making a false declaration is a form of malpractice.

Student signature: 1030371@bcas.ac

Date: 28.11.2024

General Guidelines

1. A Cover page or title page - You should always attach a title page to your assignment. Use previous page as your cover sheet and make sure all the details are accurately filled.
2. Attach this brief as the first section of your assignment.
3. All the assignments should be prepared using a word processing software.
4. All the assignments should be printed on A4 sized papers. Use single side printing.
5. Allow 1" for top, bottom, right margins and 1.25" for the left margin of each page.

Word Processing Rules

1. The font size should be 12 point, and should be in the style of Time New Roman.
2. Use 1.5 line spacing. Left justify all paragraphs.
3. Ensure that all the headings are consistent in terms of the font size and font style.
4. Use footer function in the word processor to insert Your Name, Subject, Assignment No, and Page Number on each page. This is useful if individual sheets become detached for any reason.
5. Use word processing application spell check and grammar check function to help editing your assignment.

Important Points

1. It is strictly prohibited to use text boxes to add texts in the assignments, except for the compulsory information. eg: Figures, tables of comparison etc. Adding text boxes in the body except for the before mentioned compulsory information will result in rejection of your work.
2. Avoid using page borders in your assignment body.
3. Carefully check the hand in date and the instructions given in the assignment. Late submissions will not be accepted.
4. Ensure that you give yourself enough time to complete the assignment by the due date.
5. Excuses of any nature will not be accepted for failure to hand in the work on time.
6. You must take responsibility for managing your own time effectively.
7. If you are unable to hand in your assignment on time and have valid reasons such as illness, you may apply (in writing) for an extension.
8. Failure to achieve at least PASS criteria will result in a REFERRAL grade.
9. Non-submission of work without valid reasons will lead to an automatic RE FERRAL. You will then be asked to complete an alternative assignment.
10. If you use other people's work or ideas in your assignment, reference them properly using HARVARD referencing system to avoid plagiarism. You have to provide both in-text citation and a reference list.
11. If you are proven to be guilty of plagiarism or any academic misconduct, your grade could be reduced to A REFERRAL or at worst you could be expelled from the course.

Unit 07: Software Development Lifecycles

Assignment Brief

Programme Title	HND in Computing
Student Name/ID Number	MN. Sihanas / 1030371
Unit Number and Title	Unit 7: Software Development Lifecycles
Academic Year	2024/ Batch 22
Unit Tutor	Ms. ALF. Sajeetha
Assignment Title	System for EaseShop Central (PVT) Ltd
Issue Date	24.09.2024
Submission Date	08.12.2024
Submission Format	
The submission should be in the form of an individual report written in a concise, formal business style using single spacing and font size 12. You are required to make use of headings, paragraphs and subsections as appropriate, and all work must be supported with research and referenced using Harvard referencing system. Please also provide an end list of references using the Harvard referencing system. The recommended word count is 5,000–5,500 words for the report excluding annexures, although you will not be penalised for exceeding the total word limit.	
Unit Learning Outcomes	
L01 Describe different software development lifecycles	
L02 Explain the importance of a feasibility study	
L03 Undertake a software development lifecycle	

L04 Discuss the suitability of software behavioural design techniques

Transferable skills and competencies developed

Vocational scenario

EaseShop Central PVT(Ltd) is a leading retail grocery shop located in Sri Lanka with a 5000-customer base. It sells several grocery items to the customers such as bakery, beverage, pharmacy, meat, seafood, etc. Currently, the shop uses an entirely manual system to handle day-to-day transactions of the shop. Therefore, several drawbacks and weaknesses are occurred such as inaccuracy, inefficiency, unreliability, and data redundancy. The top-level management of the shop has identified the need for a computer-based database system for handling day-to-day transactions of the shop.

As a software engineer at Spera Labs (PVT) Ltd, you're tasked with developing a comprehensive system for EaseShop Central PVT(Ltd), a prominent grocery retailer. In this scenario, EaseShop Central aims to streamline its operations by implementing a digital platform that manages customer interactions, orders, products, and categories efficiently.

Customers visiting EaseShop Central PVT(Ltd)'s digital platform will have the ability to browse through a wide array of products, select items of their choice, add them to their virtual shopping cart, proceed to make payments securely, and finally place their orders. Throughout this process, an admin, responsible for overseeing the management of customer details, product inventory, and account management, will be actively involved. The **admin's tasks** include adding new products to the system, deleting items, modifying product information as necessary, and managing customer account details such as registration information and preferences. Additionally, the admin will handle any inquiries or issues that may arise during the shopping process, ensuring a seamless and satisfactory experience for both customers and the business. By efficiently managing customer interactions and product inventory through the digital platform, EaseShop

Central PVT(Ltd) aims to enhance convenience, accuracy, and overall customer satisfaction in its retail operations.

Assignment activity and guidance

Task 01

- 1.1 Describe two iterative and two sequential software life cycle models in detail, highlighting their respective characteristics and suitability for different project types.
- 1.2 Explore the Spiral Model, outlining its distinct phases and highlighting the process of **risk management** within this flexible framework of continuous improvement.
- 1.3 Identify a suitable SDLC model for the development of the System according to the assessment and justify why it is appropriate with an example.
- 1.4 Evaluate the advantages and disadvantages of applying the Waterfall lifecycle model to a large-scale software development project.

Task 02

- 2.1 Explain the purpose of feasibility study report.
- 2.2 Analysis of software needs using requirement gathering techniques, e.g. interviews, observation, investigation of documentation, questionnaires, and focus groups. A detailed explanation of key drivers such as performance and efficiency, legacy systems upgrade, etc.
- 2.3 Prepare a detailed account of feasibility criteria and constraints, e.g., legal, social, economic, technical, timescales and organizational constraints also identification, evaluation and recommendation of solutions, with justification.
- 2.4 Assess the impact of different feasibility criteria on a software investigation

Note: The feasibility study report format has been attached

Task 03

3.1 You must prepare a **system specifications report** that will explore a business need using relevant software tools and approaches. The final report must explain these requirements in a way that diverse personnel across the system development lifecycle (project manager, team lead, testers, etc.) can comprehend.

A DFD Diagram for Context and Level 1 is required; level 2 may be specified when the feature requires additional description and explanation, and level 3 may be described where the feature does not require additional detailing and explanation. ER Diagram, Class Diagram, and Use Case Diagram and activity diagram.

As a software copy distribution, an SRS IEEE Template is provided to you; you must follow the same format and structure

3.1 Analyse how software requirements can be traced throughout the software lifecycle. Critically evaluate two approaches to improve software quality and how the use of function design paradigms in the software development life cycle improves software quality, in a **report**.

Task 04

4.1. Discuss, using examples, the suitability of software behavioural design techniques and Analyse a range of software behavioural tools and techniques

4.2. Differentiate between a finite state Machine (FSM) and an extended FSM, providing an application of use for both

4.3. Present justifications of how data-driven software can improve the reliability and effectiveness of software

Recommended Resources

Please note that the resources listed are examples for you to use as a starting point in your research – the list is not definitive.

Learning Outcomes and Assessment Criteria

Pass	Merit	Distinction
LO1 Describe different software development lifecycles		
P1 Describe two iterative and two sequential software lifecycle models. P2 Explain how risk is managed in software lifecycle models	M1 Discuss using an example, why a particular lifecycle model is selected for a development environment	
LO2 Explain the importance of a feasibility study		
P3 Explain the purpose of a feasibility report. P4 Describe how technical solutions can be compared	M2 Discuss the components of a feasibility report.	
LO3 Undertake a software development lifecycle		
P5 Undertake a software investigation to meet a business need. P6 Use appropriate software analysis tools/techniques to carry out a software investigation	M3 Analyse how software requirements can be traced throughout the software lifecycle. M4 Discuss two approaches to improving software quality.	

and create supporting documentation.		
LO4 Discuss the suitability of software behavioural design techniques		
P7 Discuss, using examples, the suitability of software behavioural design techniques.	M5 Analyse a range of software behavioural tools and techniques. M6 Differentiate between a finite state machine (FSM) and an extended FSM, providing an application of use for both.	D4 Present justifications of how data-driven software can improve the reliability and effectiveness of software.

Acknowledgement

I am deeply grateful to Mrs. ALF. Sajeetha, whose valuable guidance and support were instrumental in completing this assignment on Software Development Lifecycles. I would also like to extend my appreciation to my peers and family for their constant encouragement. This project provided an excellent opportunity to apply theoretical knowledge to practical scenarios, and I am thankful for all the resources and assistance that contributed to this work.

Abstract

This report explores the development of a comprehensive digital platform for EaseShop Central PVT (Ltd), a leading grocery retailer in Sri Lanka. The project aims to address the limitations of the existing manual system, including inefficiency, inaccuracy, and data redundancy. The study examines various software development lifecycle (SDLC) models, assesses the feasibility of the proposed solution, and provides detailed system requirements using structured methodologies such as DFDs, ER diagrams, and class diagrams. The document also discusses the suitability of behavioural design techniques for software development and emphasizes the role of data-driven software in improving reliability and effectiveness. This report was prepared under the guidance of Mrs. ALF. Sajeetha, ensuring alignment with academic standards and professional practices.

Table of Contents

Introduction.....	1
1.1 Software Life Cycle Models	2
1.1.1 Iterative Models	2
1.1.2 Sequential Models	4
1.2 How risk is managed in Spiral software lifecycle model	6
1.2.1 Phases of the Spiral Model	6
1.2.2 Risk Management in the Spiral Model	7
1.3 Identifying a Suitable SDLC Model	8
1.3.2 Why Not Waterfall or V-Model?.....	13
1.3.3 Example Project Using the Agile Model.....	14
2. Feasibility report attached along with this report.	14
3. System Requirement Specification report attached along with this report.....	14
4.1 Suitability and Analysis of Software Behavioural Design Techniques and Tools	15
4.1.1 Suitability of Software Behavioural Design Techniques	15
4.1.2 Analysis of Software Behavioural Tools and Techniques.....	16
4.2 Differentiation Between Finite State Machine (FSM) and Extended FSM (EFSM).....	18
4.2.1 Finite State Machine (FSM)	18
4.2.2 Extended Finite State Machine (EFSM).....	18
4.2.3 Key Differences Between FSM and EFSM.....	19
4.2.4 FSM Application: Traffic Light Control System	20
4.2.5 EFSM Application: ATM System	21
Conclusion	22
References.....	23

List of Figures

Figure 1: Agile model steps (Gray, 2020)	2
Figure 2: spiral model workflow (Simran, 2024)	3
Figure 3: workflow of waterfall model (Khanna, 2021)	4
Figure 4: workflow of V - Model (Nakkasem, 2020)	5
Figure 5: best agile framework.	8
Figure 6: introduction of scrum	8
Figure 7: scrum artifacts	9
Figure 8: key roles in scrum.....	10
Figure 9: scrum events	11
Figure 10: scrum workflow.....	11
Figure 11: benefits of scrum	12
Figure 12: conclusion of selecting Agile's scrum framework.	13

List of Tables

Table 1: differences between FSM & EFSM.....19

Introduction

The rapid advancements in technology have transformed the way businesses operate, emphasizing the need for efficient and reliable digital solutions. EaseShop Central PVT (Ltd), a prominent grocery retailer in Sri Lanka, faces several challenges due to its reliance on manual processes for handling day-to-day transactions. These challenges include inaccuracy, inefficiency, and data redundancy, which hinder the business's ability to meet customer expectations and streamline operations.

This report aims to develop a robust digital platform tailored to the specific needs of EaseShop Central. It explores suitable software development lifecycle models, conducts a detailed feasibility study, and defines system specifications to create a seamless e-commerce solution. The proposed platform will enable customers to browse products, manage shopping carts, and place orders securely, while administrators will benefit from tools for inventory management, customer account handling, and inquiry resolution.

The following sections provide a comprehensive analysis of the system's requirements, feasibility, and design methodologies, ensuring the proposed solution aligns with the organization's operational goals and industry standards.

1.1 Software Life Cycle Models

1.1.1 Iterative Models

a. Agile Model



Figure 1: Agile model steps (Gray, 2020)

The Agile software development model follows an iterative approach where the project is divided into small increments or iterations, each of which produces a working product feature by feature. Each iteration is a mini-software project that includes all tasks of the SDLC: planning, requirements analysis, design, coding, and testing. Agile models emphasize customer collaboration, flexibility, and rapid delivery of functional software, allowing for changes in requirements even late in the development process (Beck, 2001).

Characteristics:

- Iterative and incremental development.
- Customer feedback is integrated after each iteration.
- Emphasizes collaboration between cross-functional teams.

Suitability:

Agile is suitable for projects with rapidly changing requirements, high customer involvement, or where quick product delivery is critical. It is often used in dynamic environments such as web development and mobile applications, where frequent updates and releases are necessary (Singh, 2016).

b. Spiral Model

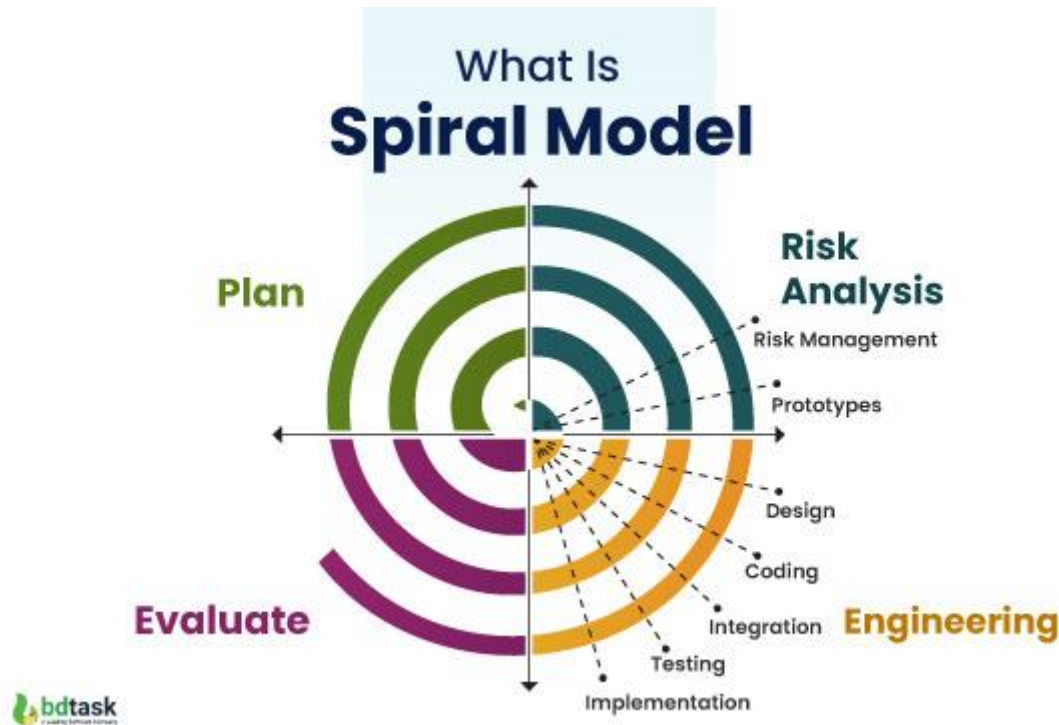


Figure 2: spiral model workflow (Simran, 2024)

The Spiral model is another iterative approach that focuses on risk analysis. This model combines the iterative nature of prototyping with elements of the Waterfall model. It allows development in a cyclical manner, where each cycle involves planning, risk assessment, engineering, and evaluation. After completing each cycle, the project progresses through refinement in a series of spirals (Boehm, 1998).

Characteristics:

- Iterative model with a focus on risk management.
- Divides the project into small segments, with risk analysis at every phase.
- Encourages user feedback at the end of each spiral.

Suitability:

The Spiral model is particularly suited for large, high-risk projects where detailed risk management is essential. It is commonly used in aerospace, defense, and critical systems where risks need to be thoroughly evaluated before moving forward (Fairley, 2003).

1.1.2 Sequential Models

a. Waterfall Model

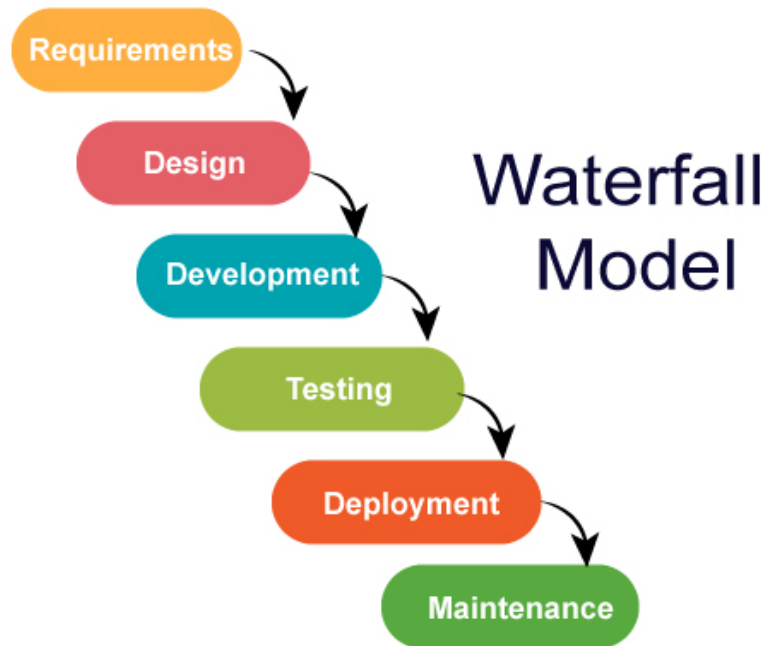


Figure 3: workflow of waterfall model (Khanna, 2021)

The Waterfall model is a traditional, linear, and sequential approach to software development. The process progresses through defined phases such as requirement gathering, design, implementation, testing, deployment, and maintenance, with each phase being completed before moving on to the next. This model follows a strict top-down approach, ensuring that no phase begins until the previous one has been fully completed (Royce, 1970).

Characteristics:

- Linear and structured process.
- Emphasizes thorough documentation.
- Requirements are fully defined before development begins.

Suitability:

The Waterfall model is suitable for projects with well-defined requirements that are unlikely to change during the development process. It is often used in industries like manufacturing and construction, where rigid adherence to a structured plan is necessary (Sommerville, 2010).

b. V-Model

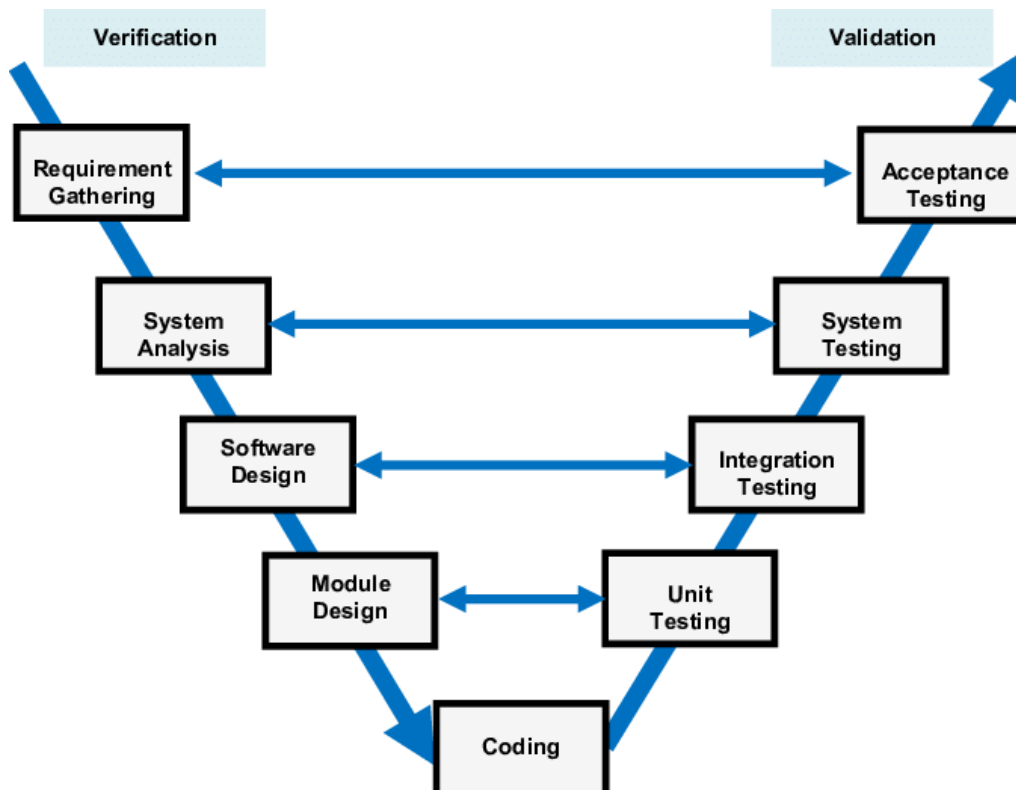


Figure 4: workflow of V - Model (Nakkasem, 2020)

The V-Model (Validation and Verification Model) is a variation of the Waterfall model but emphasizes verification and validation activities. It moves down the left side of the 'V', detailing requirements and design, and progresses back up the right side, focusing on testing at each stage. Testing phases are integrated into every corresponding development phase, providing early error detection (Assakkaf, 2014).

Characteristics:

- Sequential model with integrated testing at each phase.
- Verification and validation are performed simultaneously.
- No phase begins until the previous one is completed.

Suitability:

The V-Model is best suited for projects where high reliability is required, such as medical or automotive software, where thorough testing is crucial. It is ideal for projects with clear, unchanging requirements (Pressman, 2015).

1.2 How risk is managed in Spiral software lifecycle model

The **Spiral Model**, introduced by Barry Boehm in 1988, is an iterative software development process that emphasizes risk management as its key feature. Unlike traditional linear models like the Waterfall model, the Spiral model is adaptive, enabling continuous improvement through multiple iterations, or "spirals," in the development process. The model consists of four main phases that are repeated throughout the project life cycle, with each iteration refining the product and addressing potential risks.

1.2.1 Phases of the Spiral Model

1. Planning Phase

In the planning phase, developers identify project objectives, establish constraints, and develop a project plan. This phase includes gathering requirements and defining the system architecture. Project stakeholders are involved to ensure their needs are understood and documented early. Planning is crucial as it sets the foundation for the subsequent phases (Boehm, 1998).

2. Risk Analysis Phase

The risk analysis phase distinguishes the Spiral Model from other SDLC models. In this phase, developers identify potential project risks and assess the likelihood of each risk materializing. Common risks include budget overruns, timeline delays, technical feasibility issues, and changing requirements. This phase involves risk mitigation strategies, such as prototyping, simulations, and other experiments, to address uncertainties. By thoroughly analyzing risks at each iteration, the Spiral Model reduces the likelihood of failure in later stages (Fairley, 2003).

3. Engineering Phase

Once risks are evaluated and managed, the actual development begins in the engineering phase. Here, developers design, implement, and test the software. This phase is repeated in every spiral, building on previous versions. Prototyping is often used in this phase to validate the design and address any risks that were not identified earlier. As the product evolves, the engineering phase includes coding, integration, and verification activities (Pressman, 2015).

4. Evaluation Phase

The evaluation phase provides an opportunity for stakeholders, including customers, to review the progress of the system. Feedback is collected and incorporated into the next iteration, allowing for continuous refinement. This phase ensures that customer requirements are being met and provides a check on the overall system's viability before proceeding further into the next spiral.

1.2.2 Risk Management in the Spiral Model

The **central tenet** of the Spiral Model is its focus on **risk management**. In each iteration, risk identification, assessment, and mitigation play a crucial role in determining the direction of the project. The process is as follows:

- **Risk Identification:** Each iteration starts by identifying potential risks, including both internal (technical complexity, resource constraints) and external (market changes, customer needs) risks.
- **Risk Assessment:** Once risks are identified, they are ranked based on their potential impact on the project. High-priority risks are addressed first. For example, technical feasibility risks may lead to early prototyping to test whether the technology selected can meet the system's requirements.
- **Risk Mitigation:** After risks are ranked, developers design mitigation strategies. This may involve creating prototypes, conducting simulations, or organizing additional stakeholder reviews. If risks cannot be mitigated at an early stage, the project might be re-planned, and scope adjustments are made accordingly (Fairley, 2003).
- **Risk Monitoring and Control:** Risk management is continuous. As new risks emerge in later iterations, previous risks may also be re-evaluated to ensure that the project remains viable and on track (Pressman, 2015). Any significant risk must be mitigated before progressing further in the life cycle, allowing the Spiral Model to reduce uncertainty and provide a structured yet flexible approach to development.

The emphasis on risk management makes the Spiral Model suitable for **large, high-risk projects** where uncertainties are frequent, such as **aerospace systems, defense projects, and complex software systems**. The ability to incorporate risk management into every phase ensures that potential issues are addressed early, increasing the likelihood of a successful outcome.

1.3 Identifying a Suitable SDLC Model

In the development of the digital platform for **EaseShop Central PVT(Ltd)**, the **Agile Model** is the most suitable SDLC approach. Given the characteristics of the project, which involve managing customer interactions, product listings, orders, and admin tasks, an iterative and flexible development approach is necessary to accommodate evolving requirements and customer needs. The Agile model is highly adaptable and allows for continuous feedback and improvements, making it a fitting choice for this environment.

1.3.1 Justification for Selecting the Agile Model

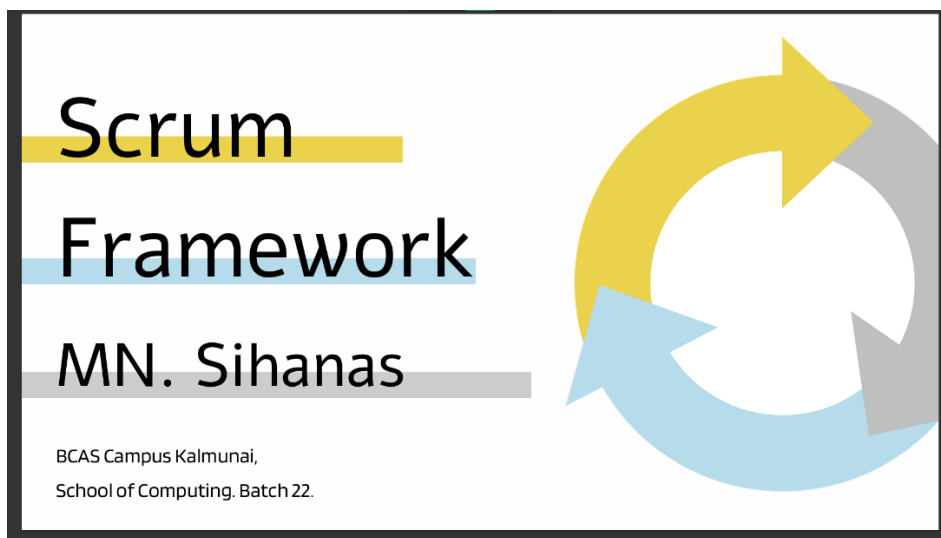


Figure 5: best agile framework.

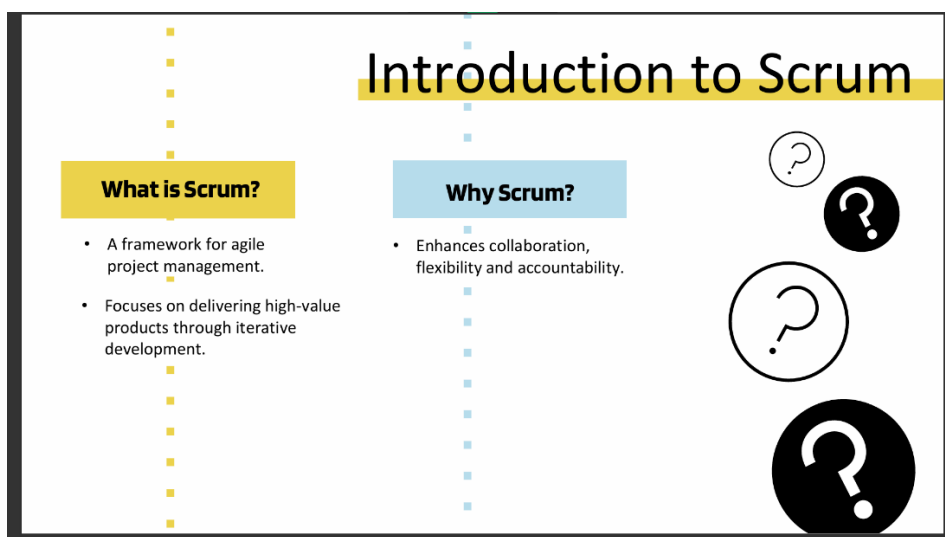


Figure 6: introduction of scrum

1. Evolving Requirements:

In the case of a digital platform like EaseShop Central's, customer preferences, product listings, and even business processes can evolve over time. The Agile model allows for iterative development, where the software is developed in small increments and feedback is integrated after each iteration. This flexibility is essential for accommodating new features, updates to the inventory system, and evolving customer interaction requirements (Beck, 2001).

- **Example:** Consider a scenario where the admin team at EaseShop Central needs to add a new product category (e.g., organic food) mid-way through the development process. With Agile, this new requirement can be incorporated into the next iteration without disrupting the project timeline. If the project were using a more rigid model, like Waterfall, such changes would be difficult to accommodate once the requirements phase was completed.



Figure 7: scrum artifacts

2. Customer and Stakeholder Involvement:

The Agile model emphasizes active collaboration with stakeholders, including customers and the admin team, throughout the development process. Since the EaseShop system will be directly interacting with customers for browsing, shopping, and placing orders, regular feedback is crucial. With Agile, stakeholders are continuously involved in reviewing the software during each sprint, ensuring that the system is aligned with business objectives and customer satisfaction goals.

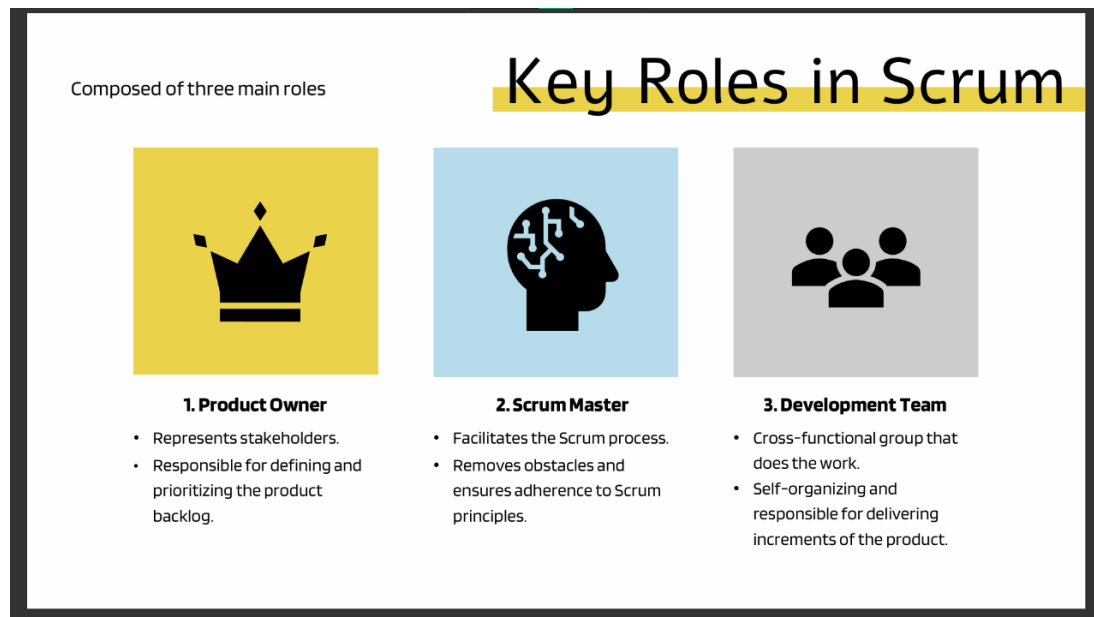


Figure 8: key roles in scrum

- **Example:** If customers report usability issues, such as difficulty in navigating the virtual shopping cart, Agile allows the development team to adjust the user interface in the next sprint. This ensures that the system is user-friendly and meets real-world customer expectations.

3. Frequent Deliveries and Flexibility:

Agile's iterative nature means that working software is delivered frequently (typically every 2-4 weeks). This is beneficial for EaseShop Central as it allows the business to gradually introduce new features and functionalities. The admin team can manage the system effectively while ensuring that no major disruptions occur to the existing manual processes.

Scrum Events



Figure 9:scrum events

Scrum Workflow

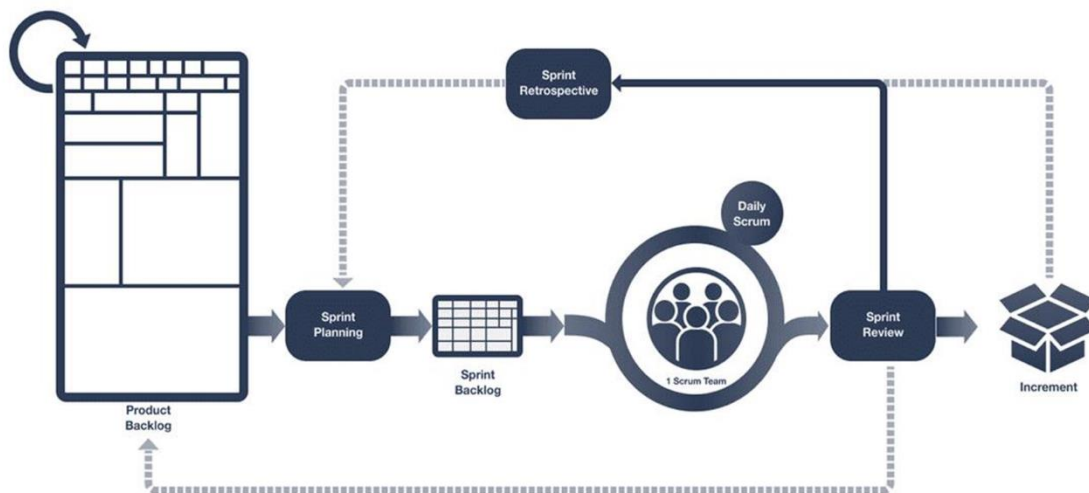


Figure 10: scrum workflow

- **Example:** The project could begin with a basic online product catalog and customer registration system, and later iterations could add functionalities like payment integration, order tracking, and inventory management. This incremental approach makes Agile ideal for handling the multiple, evolving components of the EaseShop Central platform.

4. Risk Management:

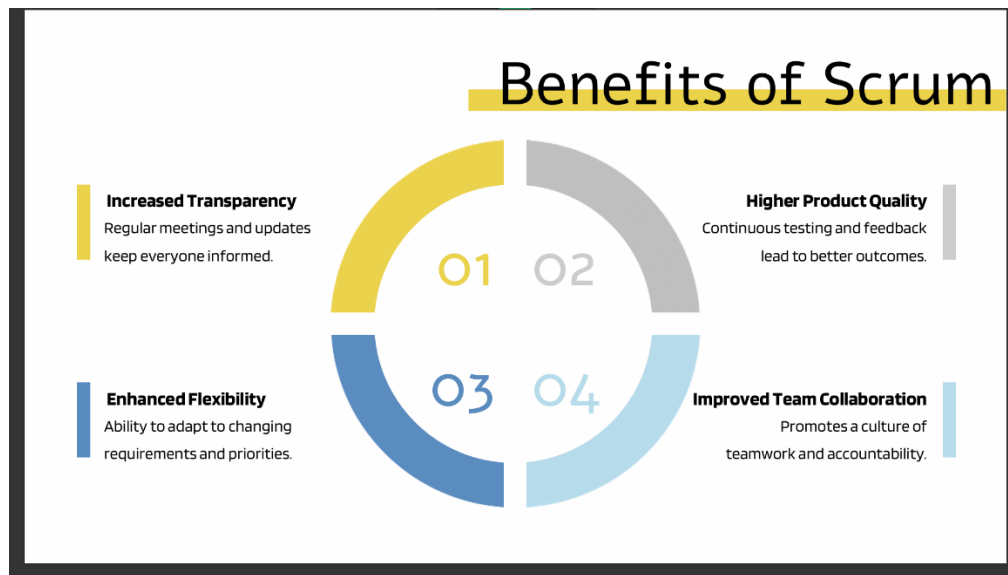


Figure 11: benefits of scrum

While Agile is not as explicitly focused on risk management as the Spiral model, it still addresses risks through continuous feedback loops and regular reassessments at the end of each sprint. This helps mitigate the risk of system failures or misaligned functionalities by ensuring that all elements are thoroughly tested and adjusted based on real-world feedback throughout development.

- **Example:** During a sprint, if the team identifies a potential issue with the security of the online payment system, this risk can be addressed immediately by prioritizing it in the next iteration. This reduces the likelihood of major security breaches later in the project.

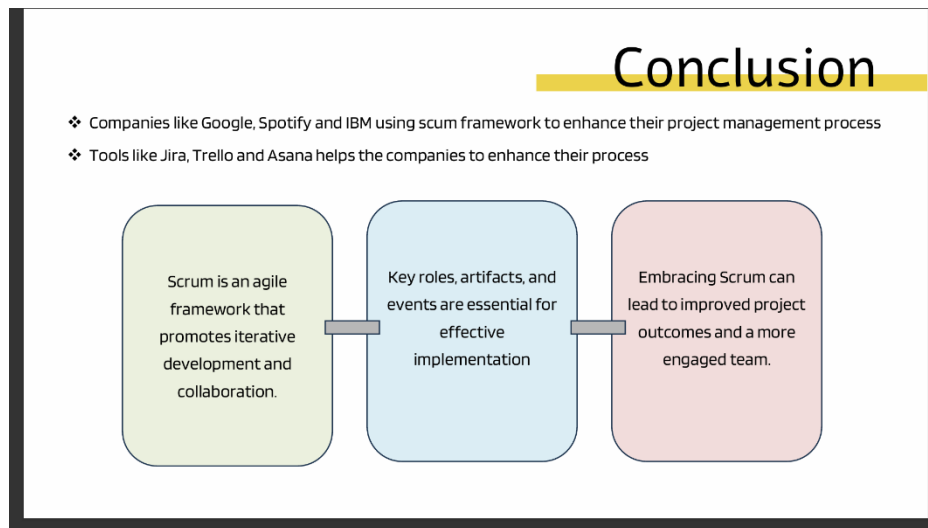


Figure 12: conclusion of selecting Agile's scrum framework.

1.3.2 Why Not Waterfall or V-Model?

Though the **Waterfall** or **V-Model** might seem attractive due to their structured and sequential nature, they are less flexible when it comes to accommodating changing requirements. In a dynamic environment like EaseShop Central, where business and customer needs might evolve, the rigid stages of the Waterfall model would not allow for such changes without significant delays. Additionally, the lack of iterative feedback would mean that crucial issues, such as customer usability or new product categories, might not be identified until the project is far into development, making adjustments more expensive and time-consuming (Royce, 1970).

On the other hand, the **V-Model** focuses heavily on verification and validation, making it suitable for environments where testing and validation are more critical than flexibility. However, for a retail platform like EaseShop Central, flexibility in requirements and constant delivery is more important than the rigorous testing approach provided by the V-Model (Assakkaf, 2014).

1.3.3 Example Project Using the Agile Model

E-commerce Platform Development:

A similar project where the **Agile model** was used effectively is the development of an **e-commerce platform** for a global retailer. In this project, the development team delivered functional iterations every two weeks, starting with a product catalog and eventually integrating features such as a shopping cart, payment gateway, and user review system. Continuous customer feedback was collected and incorporated into each sprint, ensuring that the system aligned with user needs and business goals. The flexibility of Agile allowed the project to adapt to changing market conditions, such as adding new product categories during development. This resulted in a successful, user-friendly system that supported business growth and customer satisfaction.

Similarly, for EaseShop Central, Agile would allow continuous improvements, flexibility in evolving customer preferences, and frequent deliveries, ensuring that the system meets business needs while keeping the risks minimal.

2. Feasibility report attached along with this report.

3. System Requirement Specification report attached along with this report.

4.1 Suitability and Analysis of Software Behavioural Design

Techniques and Tools

Software **behavioural design techniques** focus on understanding and modelling the dynamic aspects of a software system. These techniques illustrate how a system behaves in response to various inputs, interactions, and events. Behavioural design ensures the functional requirements of the system are met and its processes flow seamlessly.

4.1.1 Suitability of Software Behavioural Design Techniques

Behavioural design techniques are suitable for projects where system dynamics and interactions need to be thoroughly understood and visualized. The following are widely used techniques:

1. Use Case Diagrams

Use case diagrams capture the functional requirements of a system by defining interactions between external actors (users, systems) and the system.

➤ Suitability:

- Ideal for initial stages of design where functional requirements need clear representation.
- Helps identify user roles and system boundaries in projects involving multiple user types (Waykar, 2015).

- ###### ➤ Example:
- For EaseShop Central PVT(Ltd), use case diagrams can depict how customers browse products, add items to the cart, and place orders, while admins manage inventory and customer accounts.

2. State Machine Diagrams

These diagrams represent the states an object can occupy in a system and transitions between those states triggered by events.

➤ Suitability:

- Essential for systems where objects undergo distinct state changes (e.g., order processing systems).
- Particularly useful in real-time systems to model and validate object behaviors (Christian Huemer, 2015).

- Example: For an online ordering system, a state machine diagram can show states like Order Placed, Payment Confirmed, Dispatched, and Delivered.

3. Sequence Diagrams

Sequence diagrams focus on object interactions over time, showing how processes execute sequentially.

- Suitability:
 - Appropriate for modeling complex workflows involving multiple system components.
 - Helps in designing distributed systems where communication timing is critical (Al-Fedaghi, 2021).
- Example: In EaseShop Central, a sequence diagram could detail how a customer query triggers interactions between the shopping cart, inventory system, and payment gateway.

4. Activity Diagrams

These diagrams represent workflows, illustrating activities, decisions, and parallel processes within the system.

- Suitability:
 - Effective for process-driven systems with multiple decision points.
 - Useful for identifying bottlenecks in workflow automation (Étienne André, 2013).
- Example: An activity diagram can demonstrate the process of order placement, including validation of customer credentials, inventory checks, and order confirmation.

4.1.2 Analysis of Software Behavioural Tools and Techniques

1. Unified Modeling Language (UML) Tools

- **Tools:** Lucidchart, IBM Rational Rose, Visual Paradigm
- **Analysis:**
 - ✓ UML tools are highly versatile, offering support for various behavioral models like sequence and activity diagrams.
 - ✓ They allow for collaborative design and ensure standardization across teams.
 - ✓ Suitable for both small-scale and large-scale projects where clear documentation is vital (Heena, 2011).

2. Flowcharts

- **Tools:** Microsoft Visio, Creately
- **Analysis:**
 - ✓ Simple and intuitive, flowcharts are ideal for illustrating basic workflows.
 - ✓ Lack precision for complex dynamic behaviors, limiting their use in systems requiring detailed interaction modeling (Chapin, 2003).

3. Event-Driven Tools

- **Tools:** MATLAB Stateflow, Simulink
- **Analysis:**
 - ✓ These tools specialize in real-time, event-driven systems.
 - ✓ They provide powerful simulation capabilities, but their complexity makes them suitable for advanced engineering and embedded system applications (Muhammad Umer, 2018).

4. BPMN (Business Process Model and Notation)

- **Tools:** Camunda, ARIS Express
- **Analysis:**
 - ✓ Focused on process management, BPMN tools are useful for visually mapping business workflows.
 - ✓ Limited when applied to intricate software behavioral designs requiring event handling or state modeling (Mateja Bule, 2015).

Behavioural design techniques and tools are essential for ensuring that a software system behaves as intended under various conditions. The choice of technique depends on the project's complexity, system type, and the level of detail required. Combining multiple techniques, as seen in the **EaseShop Central** scenario, ensures comprehensive coverage of dynamic behaviours, enhancing the system's reliability and user experience.

4.2 Differentiation Between Finite State Machine (FSM) and Extended FSM (EFSM)

Finite State Machines (FSMs) and Extended Finite State Machines (EFSMs) are models used to represent system behaviors in terms of states and transitions. They are widely used in software and systems engineering for understanding and designing sequential logic and system interactions.

4.2.1 Finite State Machine (FSM)

An FSM is a model consisting of a finite number of states, transitions between these states, and inputs/events that trigger these transitions. Each state represents a unique condition of the system.

Key Characteristics:

- **State-Based Transitions:** Transitions occur strictly based on inputs or events.
- **Deterministic:** FSMs are often deterministic, meaning a given state and input always lead to a predictable next state.
- **Simple Memory:** FSMs have no additional memory apart from the current state (Francesco Mondada, 2018).

Applications:

- **Traffic Light System:** An FSM can represent the transitions between states like **Red**, **Yellow**, and **Green** based on a timer input.

4.2.2 Extended Finite State Machine (EFSM)

An EFSM extends the FSM by incorporating **variables** and **conditions** alongside states and transitions. This allows for more complex behavior modeling.

Key Characteristics:

- **State-Based with Conditions:** Transitions depend not only on inputs but also on conditions evaluated using variables.
- **Memory-Dependent:** EFSMs maintain variables, which enable context-aware transitions.
- **Complex Transitions:** Supports actions and guards (logical conditions) for transitions (Vladimir I. Ulyantsev, 2012).

Applications:

- **ATM System:** An EFSM can model an ATM, where transitions (e.g., from "Idle" to "Transaction") depend on conditions like the user entering a correct PIN or sufficient account balance.

4.2.3 Key Differences Between FSM and EFSM

Aspect	Finite State Machine (FSM)	Extended Finite State Machine (EFSM)
Memory	No additional memory apart from the current state.	Includes variables to store additional information.
Transitions	Triggered solely by inputs or events.	Triggered by inputs and evaluated conditions.
Complexity	Simpler and suitable for systems with limited states.	Handles complex logic and conditions efficiently.
Usage Scope	Small-scale systems with clear transitions.	Large-scale systems requiring context-awareness.

Table 1: differences between FSM & EFSM

4.2.4 FSM Application: Traffic Light Control System

Use in Traffic Management:

Traffic lights are essential for ensuring orderly vehicle movement at intersections. The FSM approach is ideal because:

- The system only needs to know the **current state** (Red, Yellow, or Green).
- Transitions are triggered solely by **timed inputs**, making it deterministic and predictable.

FSM Design:

1. States:

- ✓ **Red:** Vehicles must stop.
- ✓ **Yellow:** Vehicles prepare to stop.
- ✓ **Green:** Vehicles can proceed.

2. Transitions:

- ✓ From **Red** to **Green**: After a set timer duration.
- ✓ From **Green** to **Yellow**: After another timer duration.
- ✓ From **Yellow** to **Red**: After a short warning period.

FSM is effective here due to the simplicity of state transitions. No external variables or conditions are required, as the states are purely dependent on timing. The system ensures a smooth and predictable flow of traffic (massawa, 2022).

4.2.5 EFSM Application: ATM System

Use in Banking Transactions:

Automated Teller Machines (ATMs) manage user transactions such as withdrawals, deposits, and balance inquiries. The EFSM is more suitable than FSM here because:

- The system's behavior depends on **conditions** like user authentication and account balance.
- EFSM can handle **dynamic transitions** based on these variables.

EFSM Design:

1. States:

- ✓ **Idle:** The ATM awaits user interaction.
- ✓ **Authenticate:** The user provides card and PIN.
- ✓ **Transaction:** The user performs actions like withdrawal or deposit.
- ✓ **Logout:** The user session ends, and the machine resets.

2. Transitions:

- ✓ From **Idle** to **Authenticate**: Triggered by inserting a card.
- ✓ From **Authenticate** to **Transaction**: Only if the PIN is correct.
- ✓ From **Transaction** to **Logout**: After completing the selected task.

3. Conditions:

- ✓ PIN validation ensures secure access to user accounts.
- ✓ Withdrawal transitions depend on conditions such as **sufficient account balance**.

The EFSM model handles the complexity of real-world conditions effectively, making it perfect for a context-sensitive system like an ATM. Using variables for account balance, PINs, and transaction limits ensures robust and adaptable behavior (Abdul Salam Kalaji, 2011).

FSMs are ideal for modelling systems with simple state transitions based purely on inputs or events. EFSMs, on the other hand, offer enhanced functionality for complex systems where transitions depend on additional conditions and variables. By leveraging these models appropriately, developers can effectively manage system behaviour across various applications.

Conclusion

The development of a digital platform for EaseShop Central PVT (Ltd) is both necessary and feasible, as demonstrated by the findings of this report. The system addresses operational inefficiencies, enhances customer satisfaction, and complies with legal and regulatory requirements. By implementing structured software development lifecycle models and leveraging behavioural design techniques, the platform ensures reliability, scalability, and ease of use.

The analysis and system specifications provided herein serve as a foundation for transforming EaseShop Central's operations, aligning the business with contemporary technological standards. Moving forward, the successful deployment of this platform will not only streamline internal processes but also position the organization as a leader in Sri Lanka's competitive retail market.

References

- Abdul Salam Kalaji, R. M. H. S. S., 2011. *science direct*. [Online]
Available at: <https://www.sciencedirect.com/science/article/abs/pii/S095058491100139X>
[Accessed 24 11 2024].
- Al-Fedaghi, S., 2021. UML Sequence Diagram: An Alternative Model. *International Journal of Advanced Computer Science and Applications*, 12(5), pp. 637-644.
- Assakkaf, I. & B. D., 2014. The Verification and Validation Process of Software Development in the Automotive Industry. *International Journal of Computer Applications*, 87(19), pp. pp.12-16.
- Beck, K. B. M. v. B. A. C. A. C. W. F. M. G. J. H. J. H. A. J. R. & K. J., 2001. *Agile Manifesto*. [Online]
Available at: <https://agilemanifesto.org/>
[Accessed 20 10 2024].
- Bhatia, A., 2010. *Project Management.com*. [Online]
Available at: <https://www.projectmanagement.com/articles/229012/Feasibility-Studies-A-Guide-to-Successful-Project-Management>
[Accessed 21 10 2024].
- Boehm, B., 1998. A Spiral Model of Software Development and Enhancement. *ACM SIGSOFT Software Engineering Notes*, 11(4), pp. pp.14-24.
- Chapin, N., 2003. *acm digital library*. [Online]
Available at: <https://dl.acm.org/doi/abs/10.5555/1074100.1074406>
[Accessed 23 11 2024].
- Christian Huemer, G. K. M. S. M. S., 2015. *research gate*. [Online]
Available at: https://www.researchgate.net/publication/302287317_The_State_Machine_Diagram
[Accessed 22 11 2024].
- Étienne André, C. C. G. R., 2013. *research gate*. [Online]
Available at:
https://www.researchgate.net/publication/257411189_Activity_Diagrams_Patterns_for_Modeling_Business_Processes
[Accessed 23 11 2024].
- Fairley, R. & W. M., 2003. Iterative Life Cycle Models and the Global Positioning System. *ACM Computing Surveys*, 35(3), pp. pp.379-412.
- Francesco Mondada, M. B.-A., 2018. *research gate*. [Online]
Available at: https://www.researchgate.net/publication/320674619_Finite_State_Machines
[Accessed 23 11 2024].
- Gray, S., 2020. *medium*. [Online]
Available at: <https://serenagray2451.medium.com/what-is-the-agile-methodology-in-software-development-c93023a7eb85>
[Accessed 20 11 2024].
- Heena, R., 2011. *acm digital library*. [Online]
Available at: <https://dl.acm.org/doi/10.1145/2007052.2007053>
[Accessed 23 11 2024].
- Kendrick, T., 2015. *Identifying and Managing Project Risk: Essential Tools for Failure-Proofing Your Project*. 3 ed. New York: AMACOM.

Khanna, D., 2021. *medium*. [Online]
 Available at: <https://luciferrocks.medium.com/introduction-to-waterfall-model-of-sdlc-software-development-life-cycle-a01c7ec5e2ef>
 [Accessed 21 11 2024].

Kumar, S., 2017. Project Feasibility Study: Key Factors for Success. *Journal of Project Management*, 10(2), pp. pp.12-20.

massawa, 2022. *all about cicuits*. [Online]
 Available at: <https://forum.allaboutcircuits.com/threads/traffic-light-fsm.186401/>
 [Accessed 24 11 2024].

Mateja Bule, G. J. G. P. M. H., 2015. *research gate*. [Online]
 Available at:
https://www.researchgate.net/publication/277634419_Business_Process_Model_and_Notation_The_Current_State_of_Affairs
 [Accessed 24 11 2024].

Muhammad Umer, M. O. M. K. L. H. B. M., 2018. Smart Power Tools: An Industrial Event-Driven Architecture Implementation. *science direct*, Volume 72, pp. 1357-1361.

Nakkasem, T., 2020. *medium*. [Online]
 Available at: <https://medium.com/software-engineering-kmitl/v-model-3a71622b3d82>
 [Accessed 24 11 2024].

Pressman, R., 2015. *Software Engineering: A Practitioner's Approach*. 8 ed. New York: McGraw-Hill.

Royce, W., 1970. Managing the Development of Large Software Systems. *IEEE WESCON*, 1(26), pp. pp.1-9.

Simran, S., 2024. *bd task*. [Online]
 Available at: <https://www.bdtask.com/blog/spiral-model-for-software-development>
 [Accessed 20 11 2024].

Singh, Y. & N. A., 2016. Agile Software Development Frameworks: Methodologies, Techniques and Tools. *International Journal of Software Engineering and Its Applications*, 10(9), pp. pp.93-108.

Sommerville, I., 2010. *Software Engineering*. 9 ed. Boston: Pearson.

Vladimir I. Ulyantsev, F. N. T., 2012. *science direct*. [Online]
 Available at: <https://www.sciencedirect.com/science/article/pii/S1474667016331561?via%3Dihub>
 [Accessed 24 11 2024].

Waykar, Y., 2015. *research gate*. [Online]
 Available at:
https://www.researchgate.net/publication/322991847_role_of_use_case_diagram_in_software_development
 [Accessed 22 11 2024].